

MINI NLP PROJECT REPORT

NLP-Based Sentiment Analysis of Movie Reviews Using TF-IDF and Logistic Regression

Problem Statement

Movie reviews contain valuable information about viewers' opinions and experiences. Manually analyzing a large number of reviews is time-consuming. The objective of this project is to develop an NLP-based system that automatically classifies movie reviews into **Positive** or **Negative** sentiment categories.

Objectives

- To analyze textual movie reviews using Natural Language Processing techniques.
- To preprocess and prepare the review text for analysis.
- To convert text data into numerical features using **TF-IDF**.
- To build a sentiment classification model using **Logistic Regression**.
- To classify movie reviews as Positive or Negative.
- To evaluate the performance of the model using accuracy, precision, recall, F1-score and confusion matrix.

Dataset

Dataset Name

IMDb Dataset of 50K Movie Reviews

Source

Kaggle

Dataset Size

- Total reviews: **50,000**
- Training data: **40,000**
- Testing data: **10,000**

Attributes

Attribute	Description
review	Text of the movie review
sentiment	Sentiment label of the review

Classes

The dataset contains two sentiment classes:

- **Positive**
- **Negative**

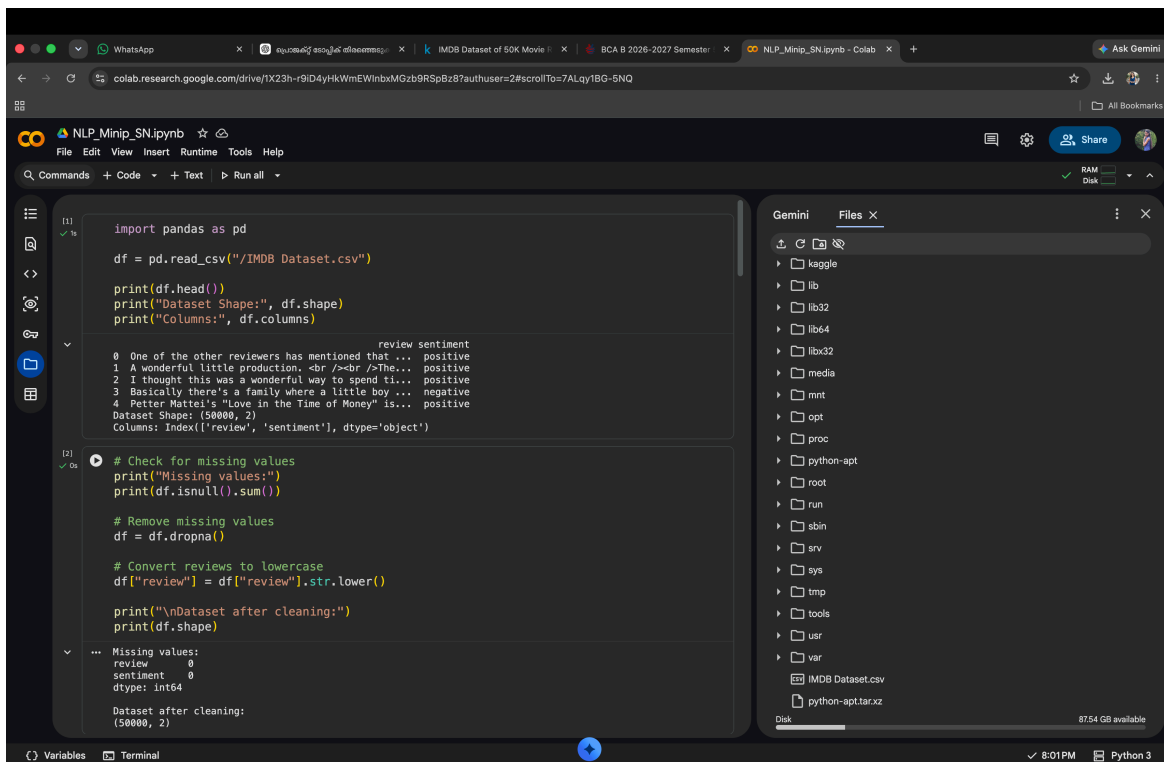
The dataset contains **25,000 positive** and **25,000 negative** reviews.

Data Preparation

The following preprocessing steps were performed:

1. Checked for missing values.
2. Removed missing values using `dropna()`.
3. Converted all review text to lowercase.
4. Separated the review text as input (X) and sentiment as target (y).
5. Converted sentiment labels into numerical values:
 - Positive → 1
 - Negative → 0

The dataset contained **no missing values**, and after cleaning it remained **50,000 records with 2 attributes**.



The screenshot shows a Google Colab notebook titled "NLP_Minip_SN.ipynb". The code in the notebook performs the following steps:

```
import pandas as pd

df = pd.read_csv("/IMDB Dataset.csv")

print(df.head())
print("Dataset Shape:", df.shape)
print("Columns:", df.columns)
```

The output of the first cell shows the first five rows of the dataset:

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

```
Dataset Shape: (50000, 2)
Columns: Index(['review', 'sentiment'], dtype='object')
```

The second cell checks for missing values and removes them:

```
# Check for missing values
print("Missing values:")
print(df.isnull().sum())

# Remove missing values
df = df.dropna()

# Convert reviews to lowercase
df["review"] = df["review"].str.lower()

print("\nDataset after cleaning:")
print(df.shape)
```

The output of the second cell shows the missing values and the dataset shape after cleaning:

```
Missing values:
review      0
sentiment   0
dtype: int64

Dataset after cleaning:
(50000, 2)
```

The right sidebar shows the file explorer with the following files and folders:

- kaggle
- lib
- lib32
- lib64
- libx32
- media
- mnt
- opt
- proc
- python-apt
- root
- run
- sbin
- srv
- sys
- tmp
- tools
- usr
- var
- IMDB Dataset.csv
- python-apt.tar.xz

The bottom status bar shows the time as 8:01 PM and the Python version as Python 3.

Methodology

The project follows the following workflow:

Movie Reviews → Data Cleaning → Train/Test Split → TF-IDF Feature Extraction → Logistic Regression → Prediction → Evaluation

1 Data Cleaning

The movie reviews were converted to lowercase and missing values were checked and removed.

2 Train-Test Split

The dataset was divided into:

- **80% training data** – 40,000 reviews
- **20% testing data** – 10,000 reviews

3 TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) was used to convert textual reviews into numerical feature vectors.

The model used a maximum of **5,000 features** and English stop words were removed.

4 Logistic Regression

Logistic Regression was used as the classification algorithm. It was selected because it is a simple and effective machine-learning algorithm for binary text classification and is suitable for a project that needs to be easily understood and explained.

The assignment also specifically lists Logistic Regression and TF-IDF among suitable approaches and emphasizes that a simple, well-understood solution is preferable to an unnecessarily complex model.

5 Prediction

The trained model predicts whether a new movie review expresses a **Positive** or **Negative** sentiment.

Implementation

Programming Language

Python

Libraries Used

- Pandas
- NumPy
- Scikit-learn
- Matplotlib

NLP Technique

TF-IDF Feature Extraction

Machine Learning Algorithm

Logistic Regression

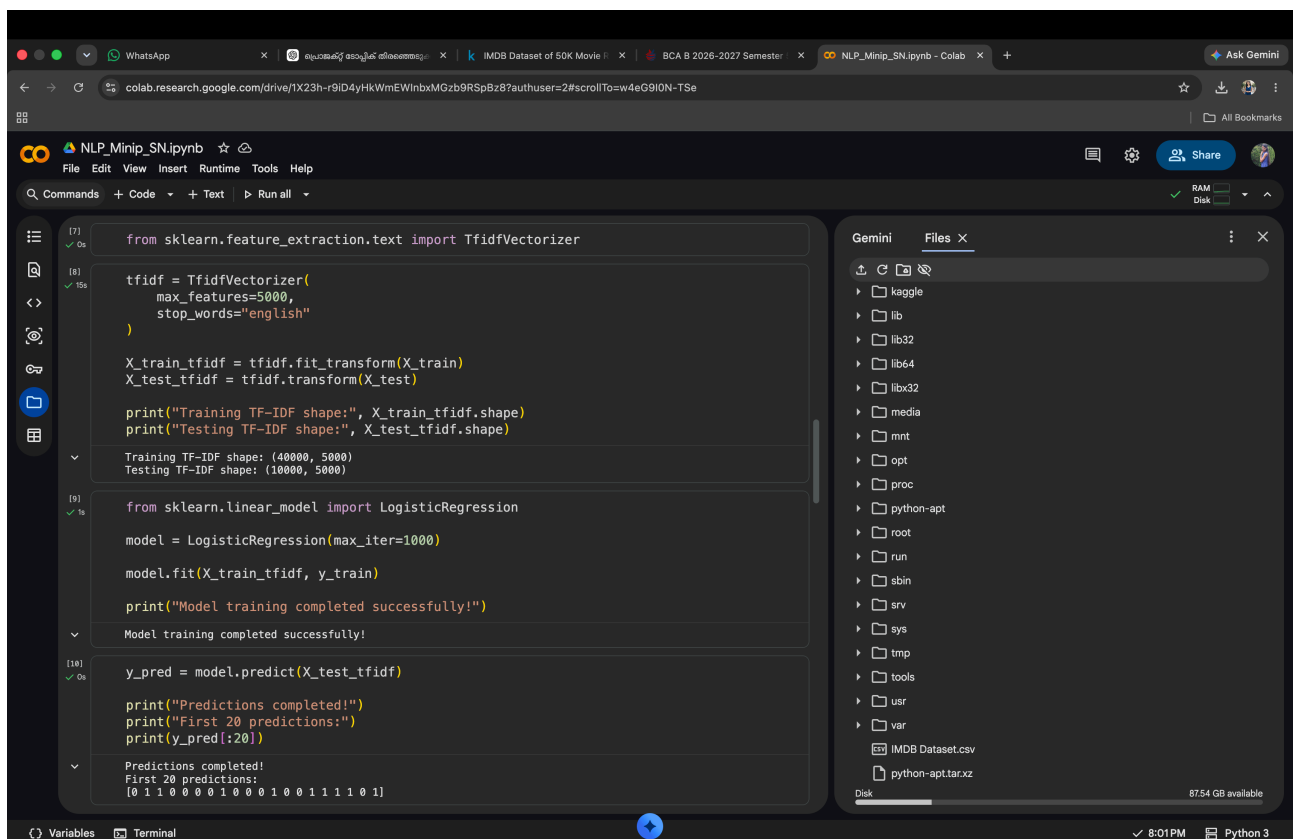
Important Implementation Details

The `TfidfVectorizer` was used with:

- `max_features = 5000`
- `stop_words = "english"`

The Logistic Regression model was trained using the TF-IDF-transformed training data. The trained model was then used to predict sentiments for the test data.

A separate prediction function was also created to classify new user-provided movie reviews.



The screenshot displays a Google Colab notebook titled "NLP_Minip_SN.ipynb". The notebook is open in a web browser, showing the code editor and the output of the cells. The code is as follows:

```
[7] from sklearn.feature_extraction.text import TfidfVectorizer

[8] tfidf = TfidfVectorizer(
    max_features=5000,
    stop_words="english"
)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("Training TF-IDF shape:", X_train_tfidf.shape)
print("Testing TF-IDF shape:", X_test_tfidf.shape)

Training TF-IDF shape: (40000, 5000)
Testing TF-IDF shape: (10000, 5000)

[9] from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train_tfidf, y_train)

print("Model training completed successfully!")

Model training completed successfully!

[10] y_pred = model.predict(X_test_tfidf)

print("Predictions completed!")
print("First 20 predictions:")
print(y_pred[:20])

Predictions completed!
First 20 predictions:
[0 1 1 0 0 0 0 1 0 0 0 1 0 0 1 1 1 1 0 1]
```

The output of the code is visible in the notebook's output area, showing the shapes of the TF-IDF matrices and the successful completion of model training and predictions. The first 20 predictions are displayed as a list of 0s and 1s.

Results

The model was evaluated using **Accuracy, Precision, Recall, F1-score and Confusion Matrix**, as recommended for classification projects in the assignment.

1 Accuracy

Accuracy = 88.98%

The model correctly classified approximately **88.98%** of the test reviews.

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy Percentage:", round(accuracy * 100, 2), "%")
```

... Accuracy: 0.8898
Accuracy Percentage: 88.98 %

2 Classification Report

Class	Precision	Recall	F1-Score	Support
Negative	0.90	0.88	0.89	5000
Positive	0.88	0.90	0.89	5000
Accuracy			0.89	10000
Macro Average	0.89	0.89	0.89	10000
Weighted Average	0.89	0.89	0.89	10000

```
[12] ✓ 0s from sklearn.metrics import classification_report

print(classification_report(
    y_test,
    y_pred,
    target_names=["Negative", "Positive"]
))
```

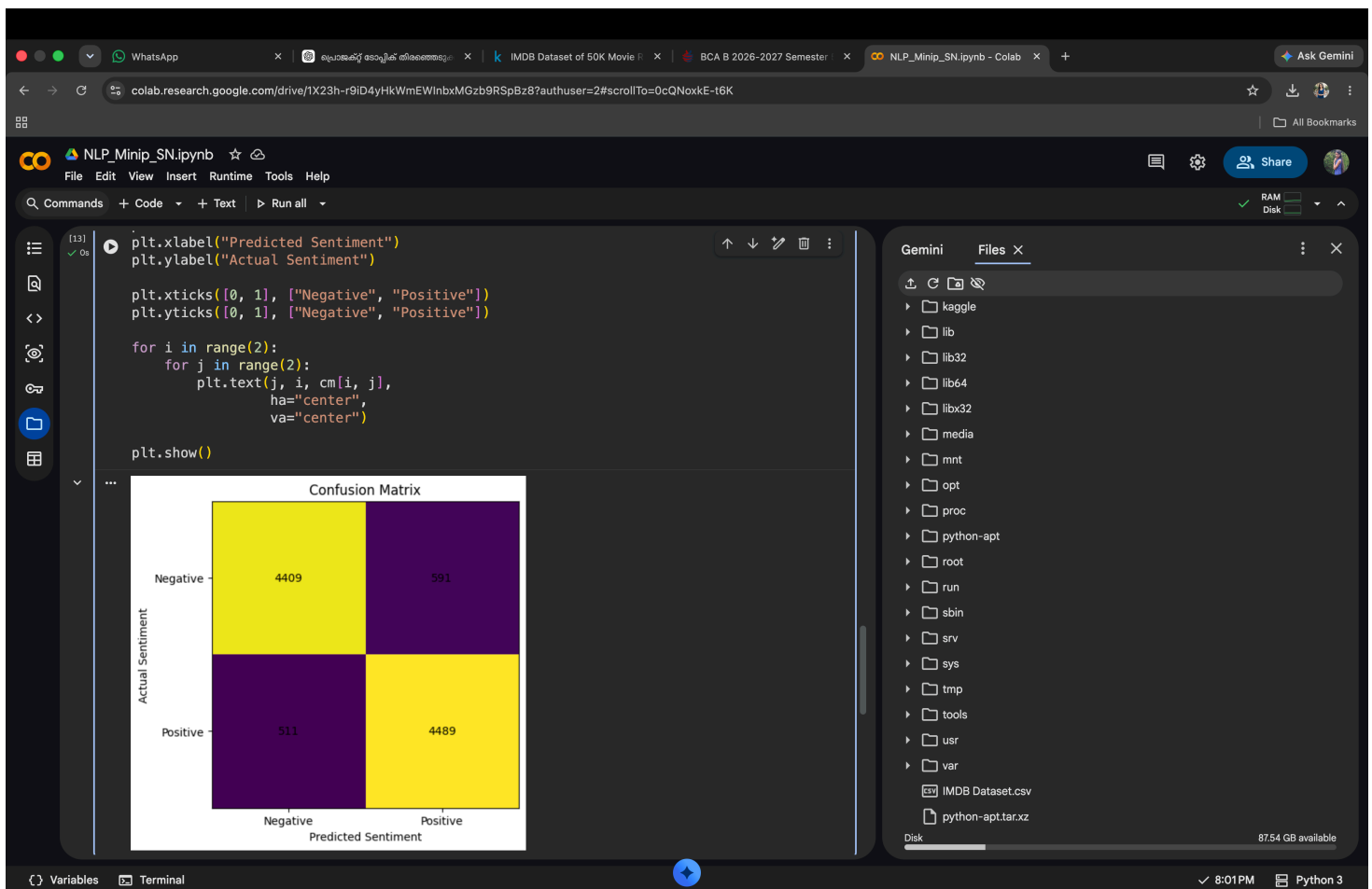
	precision	recall	f1-score	support
Negative	0.90	0.88	0.89	5000
Positive	0.88	0.90	0.89	5000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

3 Confusion Matrix

The confusion matrix produced the following results:

	Predicted Negative	Predicted Positive
Actual Negative	4409	591
Actual Positive	511	4489

The model correctly classified **4409 negative reviews** and **4489 positive reviews**.



4 Sample Predictions

The system was tested with new reviews.

Input:

This movie was amazing and I really enjoyed it.

Predicted Sentiment: Positive

Input:

This movie was boring, terrible and a complete waste of time.

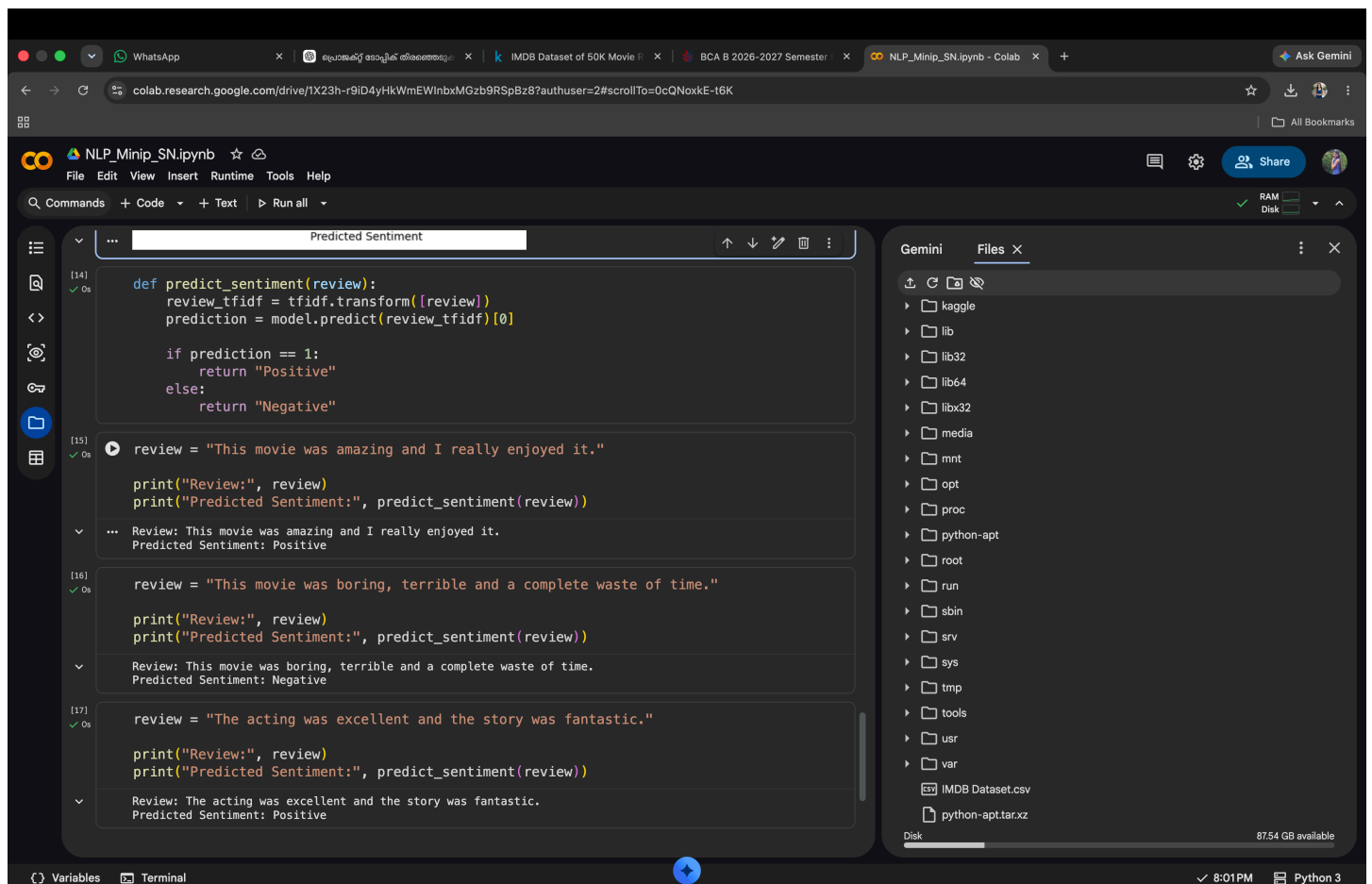
Predicted Sentiment: Negative

Another positive review was also correctly classified:

The acting was excellent and the story was fantastic.

Predicted Sentiment: Positive

The assignment recommends including metrics, tables, graphs, sample outputs and screenshots in the Results section, so your screenshots can be placed here.



The screenshot displays a Google Colab notebook titled "NLP_Minip_SN.ipynb". The notebook contains three code cells, each followed by its output. The first code cell defines a function `predict_sentiment(review)` that uses TF-IDF and a pre-trained model to predict sentiment. The second code cell tests the function with the review "This movie was amazing and I really enjoyed it.", resulting in a "Positive" prediction. The third code cell tests the function with the review "This movie was boring, terrible and a complete waste of time.", resulting in a "Negative" prediction. The fourth code cell tests the function with the review "The acting was excellent and the story was fantastic.", resulting in a "Positive" prediction. The right sidebar shows the file explorer with various folders and files, including "IMDB Dataset.csv". The bottom status bar indicates the system is running Python 3 at 8:01 PM.

```
[14] def predict_sentiment(review):
      review_tfidf = tfidf.transform([review])
      prediction = model.predict(review_tfidf)[0]

      if prediction == 1:
          return "Positive"
      else:
          return "Negative"

[15] review = "This movie was amazing and I really enjoyed it."

print("Review:", review)
print("Predicted Sentiment:", predict_sentiment(review))

Review: This movie was amazing and I really enjoyed it.
Predicted Sentiment: Positive

[16] review = "This movie was boring, terrible and a complete waste of time."

print("Review:", review)
print("Predicted Sentiment:", predict_sentiment(review))

Review: This movie was boring, terrible and a complete waste of time.
Predicted Sentiment: Negative

[17] review = "The acting was excellent and the story was fantastic."

print("Review:", review)
print("Predicted Sentiment:", predict_sentiment(review))

Review: The acting was excellent and the story was fantastic.
Predicted Sentiment: Positive
```

Limitations

- The model is trained specifically on movie reviews, so its performance may not be the same on other types of text.
- TF-IDF mainly represents words statistically and may not fully understand the context of a sentence.
- The system may have difficulty understanding **sarcasm**, irony or complex expressions.
- The model may produce incorrect predictions for reviews containing unfamiliar words or expressions.
- The project uses only two sentiment categories: Positive and Negative.

Future Scope

The project can be improved in the future by:

- Using a larger and more diverse dataset.
- Applying advanced text preprocessing techniques.
- Exploring deep-learning models such as **RNN, LSTM and GRU**.
- Using transformer-based models such as BERT.
- Supporting multiple languages.
- Developing a web or mobile application for real-time sentiment prediction.
- Improving the evaluation using additional experiments and models.

These types of improvements are consistent with the assignment's suggested future-scope areas such as larger datasets, advanced models, multilingual support and web/mobile deployment.

Conclusion

This project developed an **NLP-based sentiment analysis system for movie reviews** using **TF-IDF and Logistic Regression**. The movie review text was cleaned and converted into numerical features using TF-IDF. Logistic Regression was then trained to classify reviews into Positive and Negative categories.

The model achieved an **accuracy of 88.98%**, with an overall F1-score of **0.89**. The confusion matrix and classification report showed that the model performed well on both sentiment classes.

Through this project, the practical application of NLP, text preprocessing, TF-IDF feature extraction, machine-learning classification and model evaluation was demonstrated.